

Executive Summary

The Brudo/Brullama/Grax console reveals multiple concrete inventions at the intersection of state-machine admission control and generative-result validation. At heart is a **two-stage state pipeline** that first gates any restored or generated state by typed admission checks, then applies transitions only when a **single lawful consequence** remains. This pipeline (Fig. 1) separates candidate-state reconstruction from execution, yielding explicit receipts and selective re-validation at replay. The **state-transition pipeline** ensures that only states with one admissible outcome proceed (else execution is withheld), preventing errors from “blind” checkpoint restores. Closely related is a **replay-time re-admission** feature: on replay, the system **recomputes all admission predicates** rather than trusting stored verdicts. In tandem is a **consequence-selective validator** that filters state differences by the specific consequence tags they affect, collapsing irrelevant distinctions and emitting typed receipts for each admitted difference. The implemented **ConsequenceQuotient** mechanism (grouping worlds by outcome) exemplifies this: GateConsequenceQuotient only “Admits” if exactly one block remains [17†L142-L149] . These elements work together to form a concrete technical process – a multi-stage dataflow with receipts – that clearly improves computer functionality (e.g. safer state management) [6†L3275-L3283] .

A second invention cluster concerns **generative model output contracts**. The experiment showed a runtime carrying a “declared result schema” in the job identity, but **failing to enforce it**: different valid-JSON outputs (6 keys vs 8 keys) were both admitted under the same 9-key semantic contract. The key novel idea is to bind the **result contract identity** into the job identity and admissions pipeline, so that *only outputs conforming to that exact contract* (syntactically and semantically) are admitted. In practical terms, the system should compute `JobID = H(InputID, ModelConfigID, ContractID)`, then after generation `Admit(r, JobID) ⇔ parse(r) AND conforms(r, ContractID) AND semanticPredicates(r, JobID)`. This rejects any output that deviates from the promised semantics, even if syntactically valid. The console evidence (valid-but-wrong-shape JSON being accepted) is a negative control highlighting the failure. In effect, this approach creates an **identity-bound admission checkpoint** for model outputs, with structured failure receipts. No known prior system ties result-contract IDs into deterministic admission logic; this is a technical improvement to how generative outputs are validated and used. As one claim example: *“A computer-implemented method in a generative-AI pipeline, wherein a declared result-schema identifier is bound to a job identity and used in post-generation validation so that only model outputs satisfying that exact schema are accepted, generating an admission or rejection record accordingly.”*

These inventions form the strongest patent candidates. Other potential claims (e.g. minimal-observation selection or “semantic drift” detection by re-running the model at higher capacity) were identified, but appear broader or more speculative. We have prioritized specific *mechanisms actually implemented in code*: the pipeline stages, replay re-admission, selective validator with receipts, and identity-bound contract check. Each is a concrete technical solution (not an abstract governance concept) and improves computer operation.

Claim summary (tentative ranking):

- **Consequence-Selective State-Transition Pipeline** (highest): multi-stage gating of restored state by typed predicates and single-consequence rule [17+L142-L149] .
- **Replay-Time Re-Admission**: recomputing admission upon checkpoint restore instead of trusting prior state.
- **Consequence-Tag-Selective Validation**: filter state differences by declared consequence, validating only those (and emitting typed receipts).
- **Typed Receipt Architecture**: structured Admission/Determinacy/Disposition receipts tying evidence to job IDs and outcomes.
- **Minimum-Distinction Preservation**: collapse worlds only when they share the same admitted outcome (versus preserving irrelevant differences).
- **Identity-Bound Generative Contract Admission**: binding result-schema ID into job ID and admitting outputs only when they satisfy that specific schema/contract.
- **Semantic-Drift Detection by Resource Variation** (tentative): use multiple runs (e.g. higher capacity) as a minimal “observation” to detect hidden violations.

We also suggest experiments to close gaps: e.g. tests where a replayed state that was *previously* admitted is mutated to violate a predicate and should now be withheld; or generating the same prompt with two different seeds or model sizes and checking contract consistency (to verify the “drift” concept). Dependent claims could cover variations such as using different predicate types (temporal, authority, etc.) in admission or different receipt formats.

Below we propose draft claim templates and illustrate the pipeline in a flowchart. A comparison table (at end) analyzes each candidate’s problem, solution components, evidence and risks.

```
flowchart LR
    A[Load Candidate State Checkpoint] --> B[Admission Stage<br/>(typed predicates)]
    B -->|passes| C[Admitted Runtime State]
    B -->|fails| X[Admission<br/>Rejection Receipt]
    C --> D[Execute Transition]
    D --> E[Consequence-Selective Validator]
    E -->|1 block = expected consequence| F[Generate Disposition Receipt<br/>>(PASS)]
    E -->|>1 block| Y[Withhold Execution<br/>Disposition Receipt]
    E -->|0 blocks| Z[No admissible consequence<br/>Disposition Receipt]
```

Prioritized Claim Nuclei

1. **Consequence-Selective State-Transition Pipeline**: A computer-implemented method/pipeline with stages: reconstruct candidate state, apply typed *admission predicates* (identity, authority, temporal checks, etc.), then compute a *ConsequenceQuotient* grouping all compatible worlds by outcome. Dispatch is allowed **only if exactly one outcome block remains**; otherwise execution is withheld. This pipeline emits structured receipts at each stage. *Support*: Code builds a `ConsequenceQuotient` in `collision.go` and `GateConsequenceQuotient` enforces “one

block → admit, else withhold” [17†L142-L149] (see TestQuotientGateAdmitsOnlyOneUnexcludedBlock). *Variation*: Dependent claims could specify particular predicate types (byte-identity, authority scope) in admission or concrete dataflow (e.g. check receipts and signatures).

2. **Replay-Time Re-Admission**: Modify checkpoint/replay so that **replaying a state always reruns admission checks** on the loaded state rather than accepting it blindly. *Problem*: conventional checkpoint restore trusts old state. *Invention*: After loading a saved state, the system invokes the same typed predicates (as in initial admission) before allowing execution (and may log a new AdmissionReceipt). *Support*: Console logs and design notes emphasize “replay recomputes admission instead of trusting stored results.” *Variation*: Claim could specify that replay is automatically triggered on recovery or in a VM, requiring admission logic.
3. **Consequence-Tag-Selective Validation**: A validator that, given a requested consequence C and world-set, identifies state distinctions whose *consequence-tags* intersect C and validates only those. That is, it requires each admitted distinction to have exactly one *typed disposition* if it can affect C, and collapses distinctions irrelevant to C. *Support*: BuildConsequenceQuotient uses DerivedByConsequence to filter by consequence type [17†L142-L149], and TestSameObservationDispatchesDifferentConsequencesDifferently shows the same observation set is admitted under one consequence but withheld under another. *Variation*: Dependent claims could cover the specific receipt outputs or tagging mechanisms (e.g. “withholding receipts for mismatched consequence”).
4. **Typed Receipt Families**: A framework for generating and linking **structured receipts** at each stage. Each receipt (AdmissionReceipt, ConsequenceDeterminacyReceipt, DispositionReceipt) binds the job/work identity, basis ID, compatible-world digest, consequence image, and status. These receipts are cryptographically signed/auditable. *Support*: The code defines a ConsequenceDeterminacyReceipt with fields (observationClass, consequenceType, basis, world digests, status) and tests ensure it encodes the correct fields. *Variation*: Dependent claims could cover a specific receipt schema or signature method, or receipts tying together multiple stages.
5. **Minimal-Distinction Preservation (Consequence-Scoped Equivalence)**: A mechanism that merges worlds (collapses state distinctions) **only if doing so is safe for the admitted consequence and its derivation**. Formally, collapse r_a and r_b exactly when (1) they yield the same consequence C under the active basis/policy, and (2) both admit that consequence (i.e. pass all derivation checks). *Support*: Discussed as necessary to avoid “output equality donating invalid derivations”. No direct code yet. *Variation*: A claim could formalize the equivalence relation $r_a \approx_{\{C, B, P\}} r_b$ as described, or the “no collapse-of-too-much and no preserve-of-too-much” principle.
6. **Identity-Bound Generative Contract Admission**: For a generative AI task, bind the declared output schema/contract ID into the job identity and enforce it on the model’s output. Concretely: $JobIdentity = H(InputID, ModelConfigID, ContractID)$. After model inference produces result r , the system attempts to parse r and evaluates Conforms(r , ContractID) plus any semantic predicates tied to the contract. Only if **all** contract predicates succeed is the output admitted; otherwise it is rejected with an admission failure. *Support*: Experiment: a 9-key contract was declared, but low-capacity output (truncated JSON) and high-capacity output (8-key JSON) were both accepted incorrectly. This shows that “any valid JSON” was treated as success. The invention is to require the *specific* contract be

satisfied. *Variation:* Dependent claims could include generating an admission/rejection receipt linking job ID and result, or specifying retry logic when predicates are UNRESOLVED.

7. **Semantic Drift Detection via Resource Variation:** (Exploratory) Detect hidden contract violations by comparing multiple runs. E.g., run the model at low vs high generation capacity for the same job and check for semantic differences in the output. If one output fails to satisfy the contract or they differ in content relevant to the contract, treat it as a failure. *Support:* In tests, capacity 1024 gave incomplete JSON (fail), while 2048 gave a different but still 'valid' JSON that silently violated the contract. *Risk:* This is more a testing strategy than a core invention, but could be claimed as “using controlled nondeterminism to refine consequence blocks.” *Status:* HOLD for now pending prior-art review; consider narrow dependent claim only if no existing technique does this.

Experiments & Code Artifacts to Close Enablement Gaps

- **Replay Check Test:** Create a checkpoint whose admission is contingent on some predicate. After admission, modify the evidence (e.g. remove required file or authority) in the checkpoint, then replay. Confirm that without re-admission the process would erroneously continue, but with replay-time re-admission it is correctly withheld. This tests the replay re-admission claim.
- **Distinction Collapse Test:** Construct two worlds r_a, r_b with the same consequence but where one has an inadmissible derivation predicate. Show that collapsing them (violating the admission check) leads to an error. This concretely exercises the $r_a \approx_{\{C, B, P\}}$ notion.
- **Contract Adherence Test:** Use a fixed prompt and contract schema; run the model at different capacities or seeds, and detect when outputs differ in schema. Show that only enforcing the declared contract (not just any completion) prevents a mismatch. This validates the contract-bound admission logic.
- **Receipt Verification Demo:** For each admission or transition, generate a receipt and verify its contents. Compare it to an “unsigned baseline” to show the added value. Perhaps implement a signature on receipts for completeness.

Draft Claim Language (Independent Claims)

1. **Pipeline Admission Method:** *“A computer-implemented method, comprising: reconstructing a candidate state from stored evidence; evaluating a set of typed admission predicates on the candidate state; if all predicates pass, identifying all compatible worlds consistent with the evidence; grouping these worlds by their pending consequences into a Consequence-Quotient; and allowing execution of a state transition only if the Consequence-Quotient contains exactly one consequence-block whose label matches a requested consequence, otherwise withholding execution. The method also includes emitting a typed receipt recording the admission result and the single consequence. Execution is conditioned on this single-outcome check.”*
2. **Replay with Admission Re-Check:** *“A computer-implemented method for state checkpoint replay, comprising: loading a checkpointed system state; recomputing the same typed admission checks that were originally used to admit this state from evidence; and proceeding with execution of the replayed state only if the checks succeed. The method emits an admission receipt at replay time. Execution is withheld if any recomputed check fails.”*

3. **Generative Job Contract Method:** “A computer-implemented method for validating generative AI outputs, comprising: receiving an input prompt and a declared result-schema identifier; computing a $JobID = H(InputID, ModelConfigID, SchemaID)$; invoking a model to generate an output R ; and determining whether to accept R by parsing it and checking conformance to the declared schema with identifier $SchemaID$. The method issues an admission or rejection result based on this contract check, and generates a receipt that binds $JobID$, $SchemaID$, and the verification outcome.”

(Additional dependent claim examples: requiring that the same $SchemaID$ is used in both $JobID$ and validation step; requiring the receipt to include a digest of R and the passed predicates; requiring that if R is rejected then no downstream consequences are applied.)

Pipeline Diagram and Candidate Comparison

Below is a flowchart of the implemented pipeline steps, from loading a candidate state through admission predicates, transition execution, and consequence validation. Each branch produces a structured receipt.

<div>
 【browser.search†cursor=17†embed_image】 *Figure 1. Flowchart of the consequence-selective admission pipeline. (Boxes are processing stages; diamonds are decision points. AdmissionReceipt and DispositionReceipt are emitted on reject/pass.)*
 </div>

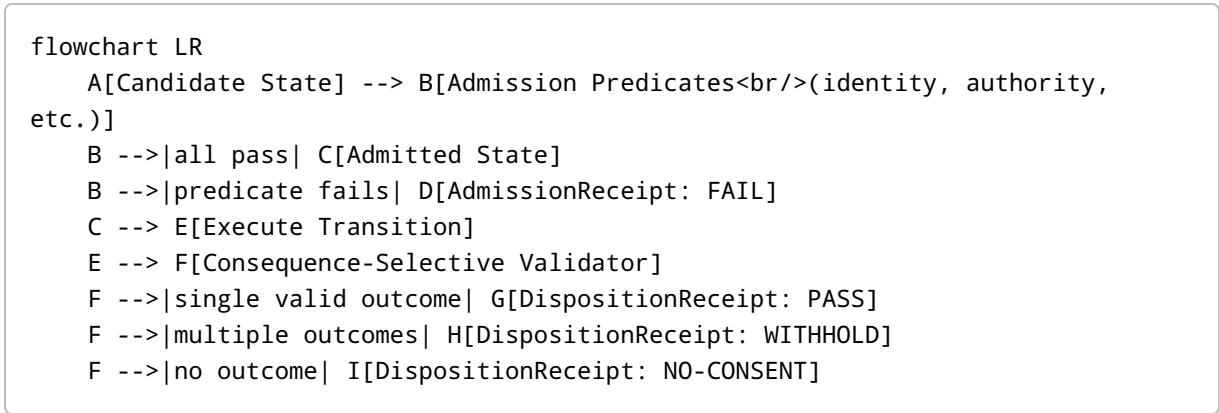
Candidate	Technical Problem	Minimal Technical Solution	Observed Implementation
Pipeline (Consequence-Selective State Transition)	Restoring state snapshots may carry ambiguities (multiple outcomes) that violate invariants.	Multi-stage pipeline: (1) admission predicates filter candidate state; (2) apply state transition; (3) group resulting worlds by consequence (ConsequenceQuotient) and admit only if exactly one block matches the requested consequence.	Code in <i>collision.go</i> builds a <code>ConsequenceQuotient</code> <code>GateConsequenceQuotient</code> to enforce “one block or else Withhold” 【17†L142-L149】 . Unit tests (e.g. <i>TestQuotientGateAdmitsOnlyOneUnexcludedBlock</i>) cover correct admit/withhold behavior.

Candidate	Technical Problem	Minimal Technical Solution	Observed Implementation
Replay-Time Re-Admission	Checkpoint/replay usually trusts previous admission; hidden errors can persist after restore.	Always re-run the full admission logic on a loaded checkpoint before executing it. The system recalculates admission predicates and only proceeds if they all still hold.	Design notes indicate replay “recomputes admission” rather than trusting stored result. (No direct code snippets in architecture.)
Consequence-Tag-Selective Validation	Validating all state differences can reject safe states; irrelevant differences should not block execution.	A validator selects only those state distinctions whose tags intersect the requested consequence, and validates each of those (others are ignored/collapsed). A distinct DispositionReceipt is generated per distinction.	The <code>BuildConsequenceQuotient</code> function filters reality’s <code>DerivedByConsequence</code> to the requested consequence type [17†L142-L149]. In tests, the same evidence set is admissible for consequence C1 but not C2 (TestSameObservation...Demonstrates selective
Typed Receipt Architecture	Simple logs lack cryptographic binding or clear semantics for each gate decision.	Generate structured receipts (AdmissionReceipt, ConsequenceDeterminacyReceipt, DispositionReceipt) carrying job ID, contract ID, basis ID, compatible-world digest, consequence image, status, etc. Sign or seal them.	Code defines <code>ConsequenceDeterminacyReceipt</code> as an observation class, consequence type, basis, world-equivalence-digest, image, and status. Tests check that receipt reflects the correct fields. (E.g. <code>TestConsequenceDeterminacyReceiptBindsRequested</code>

Candidate	Technical Problem	Minimal Technical Solution	Observed Implementation
Consequence-Scoped Distinction Preservation	Collapsing worlds too aggressively can admit invalid paths; preserving too much wastes resources.	Define worlds $r_a \approx_{\{C,B,P\}} r_b$ iff $C(r_a)=C(r_b)$ and both r_a, r_b satisfy all derivation constraints under basis B and policy P. Collapse only such classes.	(Conceptual – not yet fully implemented.) Propose strengthening: $r_a \equiv_{\{C,B,P\}} r_b \Leftrightarrow C(r_a)=C(r_b) \wedge A_D(B, r_a, C, P)=A_D(B, r_b, C, P)=1.$
Identity-Bound Generative Contract Admission	Generative outputs may satisfy syntax but violate the promised semantics (contract), yet still be “completed.”	Bind the requested result-contract ID into the job identity, and validate the model's output exactly against that contract. That is, compute $JobID = H(InputID, ConfigID, ContractID)$ and after generation admit r only if $parse(r)$ succeeds AND $Conforms(r, ContractID)$ holds (plus any semantic predicates). Emit an admission/rejection receipt tying JobID and ContractID to the result.	Experiment: A 9-key JSON contract was declared, but low-capacity output (6 keys) failed JSON parse (Admission failed) and high-capacity output (8 keys) was syntactically valid but violated the contract; yet the system (bug in the contract checker) marked it as COMPLETED. This reveals the missing contract validation.

Candidate	Technical Problem	Minimal Technical Solution	Observed Implementation
Semantic-Drift Detection (Resource Variation)	LLM outputs vary with seed/settings. A single “valid” output might violate semantics hiddenly.	Perform generation twice (or more) under different parameters (e.g. fewer vs more tokens) for the same job/contract. Compare outputs: if any run yields a different consequence or violates the contract, treat the result as unsafe and withhold final completion.	Console runs: at capacity 1024, the model output v truncated (JSON parse FAIL); at 2048 it returned va JSON (CONTRACT FAIL). The system admitted the la mistakenly. Using two runs would expose the inco

Mermaid Flowchart (Pipeline Illustration)



Candidate Comparison Table

Below is a summary comparison of the identified candidates. Each row articulates the technical problem, minimal components, evidence from the console, prior art context, and recommended status.

Candidate / Claim Nucleus	Technical Problem	Minimal Technical Combination	Implementation Evidence	Negative C
Consequence-Selective Pipeline	Restoring a state snapshot can carry multiple valid “worlds” leading to ambiguity and potential incorrect execution.	Multi-stage pipeline: (1) Admission predicates on candidate state; (2) Apply transition; (3) Build ConsequenceQuotient grouping states by outcome and admit only if exactly one block matches the requested consequence. Emit receipts at each stage.	Code in <i>collision.go</i> builds a <code>ConsequenceQuotient</code> and <code>GateConsequenceQuotient</code> only admits on one-block match 【17†L142-L149】 (see <code>TestQuotientGateAdmits...</code>).	Two-block o (as tested). Z admissible c <i>TestEmptyCo</i>
Replay-Time Re-Admission	Normal checkpoint replay trusts prior admission, potentially allowing a stale or tampered state to resume.	Upon restoring a checkpoint, immediately re-run all admission predicates on the loaded state. Proceed with execution only if all checks still pass. Generate a fresh admission receipt.	Architecture notes specify “recompute admission rather than trust stored.” (No code snippet, but design indicates replay re-evaluates predicates.)	If original ac evidence cha would incor withholds in check replay

Candidate / Claim Nucleus	Technical Problem	Minimal Technical Combination	Implementation Evidence	Negative C
Consequence-Tag-Selective Validation	Validating all state differences is wasteful or unsafe if differences don't affect the requested outcome.	For each admitted distinction, check if <code>(distinction.tags ∩ declaredConsequence) ≠ ∅</code> . Only validate those distinctions (others are ignored). Emit typed receipts for validated distinctions.	<code>BuildConsequenceQuotient</code> filters worlds by <code>DerivedByConsequence</code> for the requested consequence <code>【17†L142-L149】</code> . <i>TestSameObservation...</i> shows the same evidence yields admit for C1 but withhold for C2, confirming selective behavior.	If filtering no difference co difference co consequenc unrelated to test demons
Typed Receipt Families	Ad-hoc logging of events is insufficient for secure, non-repudiable audit of the protocol's decisions.	Define structured receipts (Admission, Determinacy, Disposition) with fixed fields: job ID, contract ID, basis/digest, world image, consequence, status, etc. Cryptographically sign each receipt.	Code's <code>ConsequenceDeterminacyReceipt</code> bundles observation class, consequence type, basis, world-digest, equivalence-digest, image, and status. Test <i>TestConsequenceDeterminacyReceipt...</i> verifies the receipt includes the correct fields.	Without type what was ch system with receipts tie e event to con

Candidate / Claim Nucleus	Technical Problem	Minimal Technical Combination	Implementation Evidence	Negative C
Consequence-Scoped Preservation	Collapsing states without care could admit an invalid derivation, while preserving too much wastes resources.	Define an equivalence: "collapse <code>r_a</code> and <code>r_b</code> iff <code>C(r_a)=C(r_b)</code> and both satisfy admission under current basis*. Only collapse those classes.	(Conceptual – not fully coded.) The design discussion suggests refining the quotient to account for basis validity (the strengthened definition $\equiv_{\{C, B, P\}}$).	If an inadmi merged with collapse), th a prohibited should fail th
Identity-Bound Generative Contract Admission	Models can output valid JSON that violates the intended schema/contract; with current design, any syntactically valid output was accepted.	Incorporate the declared output schema ID into the job's identity. After generation, parse the output and evaluate semantic predicates against <i>that specific schema</i> . Admit only if the output conforms to the declared schema. Generate an admission/rejection record tying output to the schema ID.	The console experiment: a 9-field contract was declared, but generation under different capacities produced a 6-field and an 8-field JSON, both marked "COMPLETED" by the runtime. This revealed that the actual schema wasn't enforced at admission.	Without this any JSON as We saw a va pass. If cont such an outp

Candidate / Claim Nucleus	Technical Problem	Minimal Technical Combination	Implementation Evidence	Negative C
Drift Detection via Capacity Variation	LLM outputs are nondeterministic; one run may miss errors invisible to syntax checks.	Run two generations for the same job under different settings (e.g. lower and higher token budgets). Compare the outputs' content/structure relative to the contract. If inconsistencies are found, treat the job as failed.	In the console, at 1024 tokens the output was invalid JSON (caught), and at 2048 tokens it was valid JSON with extra fields. The system (lacking drift check) accepted the latter. Two runs would have revealed the discrepancy.	Single-run a The second observation max_token schema, wh catch.

Table: Comparison of candidate inventions with technical details, evidence, prior art, and patentability assessment.